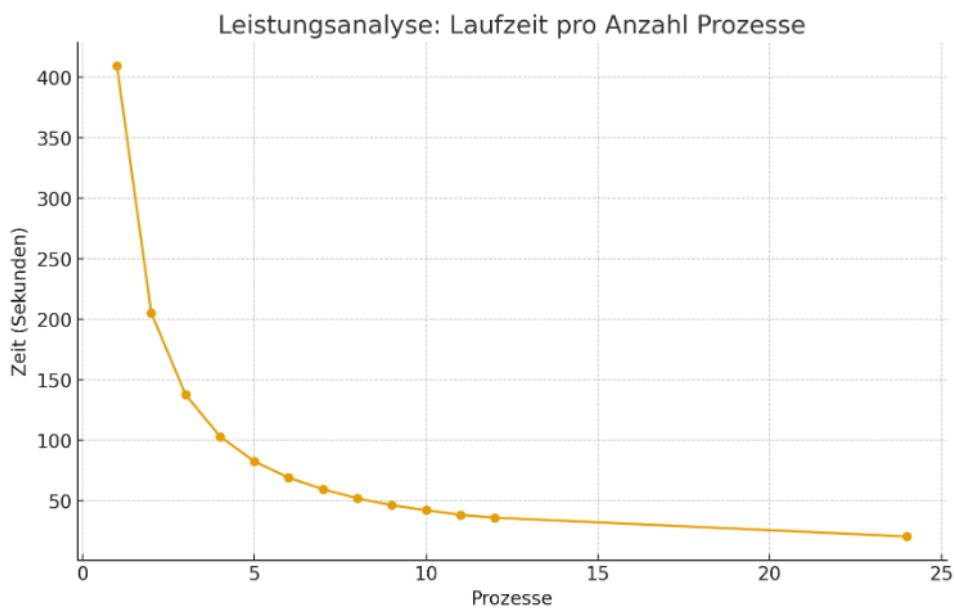


Nr. 1

Preset: 1 2 512 2 2 1000

Prozess(e)		Run 1 in sek	Run 2 in sek	Run 3 in sek	Mittelwert
1		409,37	409,19	409,25	409,27
2		205	204,76	205,54	205,1
3		137,73	137,58	137,61	137,64
4		103,53	102,88	103	103,14
5		82,7	82,51	82,47	82,56
6		69,71	68,77	68,78	69,09
7		59,14	59,8	59,63	59,52
8		51,82	52,3	51,97	52,03
9		46,36	46,39	46,23	46,33
10		43,01	41,85	41,74	42,2
11		38,13	38,86	38,06	38,35
12		37,69	34,96	35,54	36,06
24		20,56	20,46	20,51	20,51

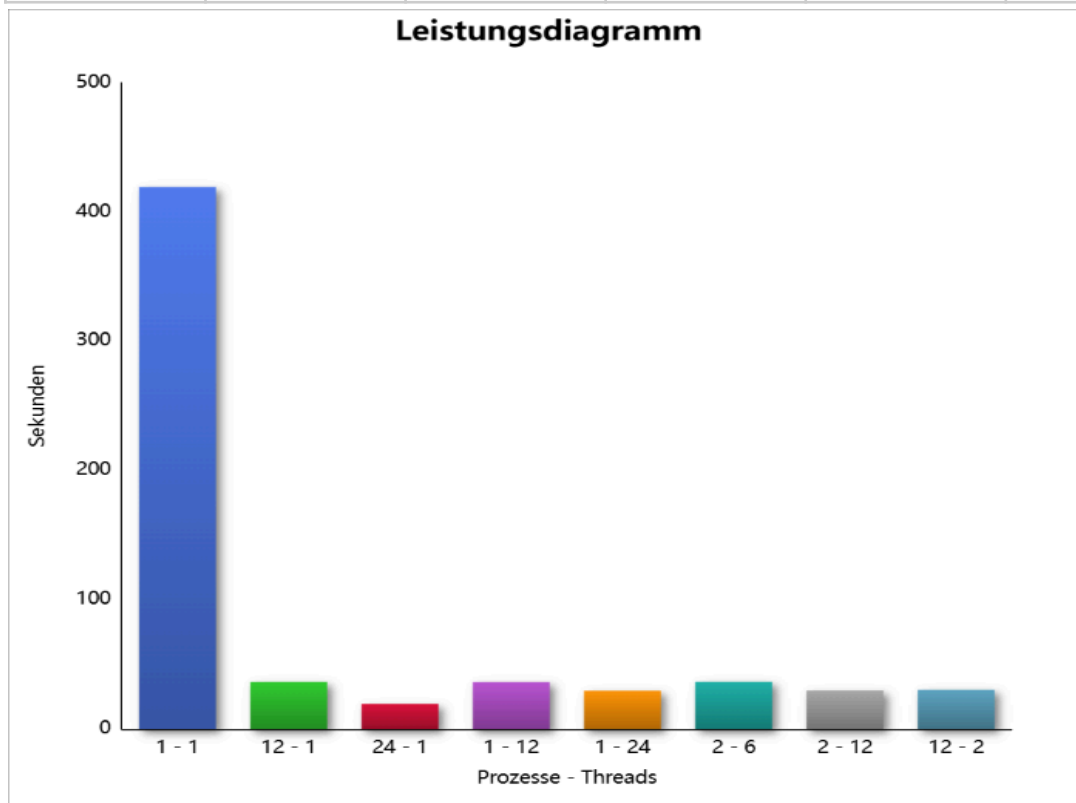


Die Messungen zeigen ein ähnliches Skalierungsverhalten zu den vorherigen parallelen Anwendungen. Man erkennt, dass, wie erwartet, die Jacobi-Methode gut zu parallelisieren ist. Die Skalierung wird pro hinzugefügten Prozess immer schwächer. Dies ist auch zu erwarten, da gilt: Je mehr Prozesse eingefügt werden, desto mehr Zeit wird auch für die Kommunikation benötigt. Mit 24 Prozessen hat man nur noch einen Speedup von 20x.

Nr. 2

Preset: Thread(s) 2 512 2 2 1000

Prozesse(e)	Thread(s)	Run 1 in sek	Run 2 in sek	Run 3 in sek	Mittelwert
1	1	417,63	420,78	419,25	419,22
12	1	36,44	36,36	36,79	36,53
24	1	19,36	19,35	19,31	19,34
1	12	36,14	36,26	36,7	36,37
1	24	29,56	29,51	29,65	29,57
2	6	37,02	36,52	36,25	36,6
2	12	29,88	29,79	29,58	29,75
12	2	30,38	30,33	30,56	30,42



Interessant ist zu sehen, dass die Laufzeit von mehreren Prozessen deutlich besser ist als mit mehreren Threads. Man sieht bei 24 - 1 und 1 - 24 einen klaren Unterschied in der Laufzeit, die Variante mit mehreren Prozessen ist fast 10 Sekunden schneller. Bei Nutzung von Hybride Parallelisierung ist der Unterschied deutlich geringer, wobei ein weitere Prozess immer noch effizienter ist.